LABORATORIO 02

Implement Programmability Objects with SQL

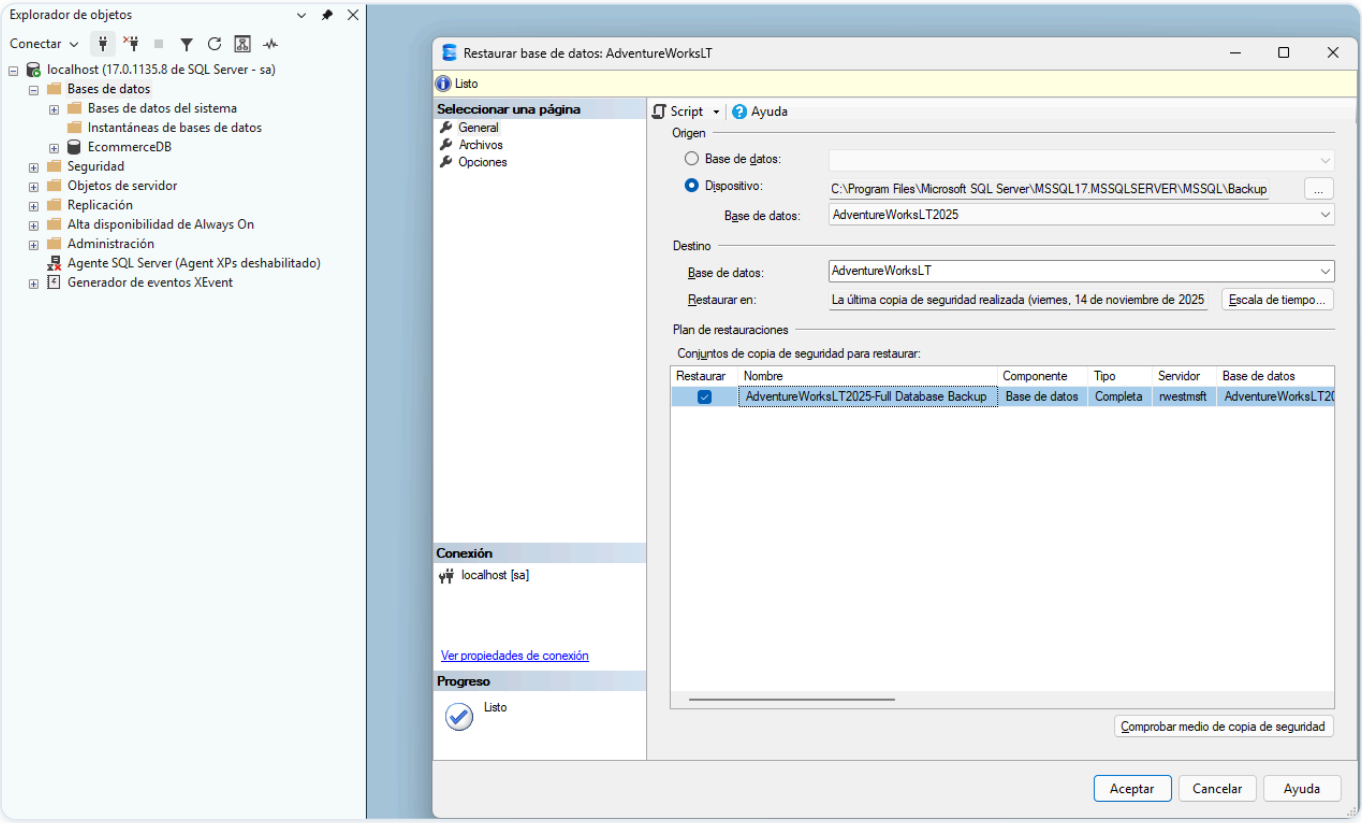
Entorno de ejecución	Microsoft SQL Server · SQL Server Management Studio (SSMS) · Windows 11
Base de datos de trabajo	`AdventureWorksLT`
Repositorio oficial	github.com/HugoAlcaide/master-labs/tree/main/lab02-programmability-objects
Fecha de realización	22 de septiembre de 2026

Descripción y Objetivos

Implementación, optimización y validación de objetos de programabilidad en SQL Server sobre la base de datos de pruebas `AdventureWorksLT`. El objetivo de este laboratorio es encapsular la lógica de negocio en el motor relacional, desacoplar el acceso directo a tablas físicas mediante capas de abstracción, garantizar la atomicidad transaccional y automatizar auditorías sin dependencia del código de la aplicación.

Prerrequisitos: Restauración de Base de Datos

Restauración del archivo de copia de seguridad oficial `AdventureWorksLT.bak` en la instancia local de SQL Server, normalizando el nombre de destino para compatibilidad con los scripts del laboratorio.



Evidencia: restauración de la base de datos AdventureWorksLT en la instancia local de SQL Server.

1. Verificación Inicial de Conectividad y Tablas Base

Comprobación de conectividad con la base de datos `AdventureWorksLT` y validación de permisos de lectura sobre las entidades clave del esquema `SalesLT` (`Customer` , `SalesOrderHeader` y `Product`).

```
USE AdventureWorksLT;
GO

SELECT TOP (5) CustomerID, FirstName, LastName
FROM SalesLT.Customer;

SELECT TOP (5) SalesOrderID, OrderDate, CustomerID
FROM SalesLT.SalesOrderHeader;

SELECT TOP (5) ProductID, Name, ListPrice
FROM SalesLT.Product;
```

RESULTADO

Resultados | Mensajes

	CustomerID	FirstName	LastName
1	1	Orlando	Gee
2	2	Keith	Harris
3	3	Donna	Carreras
4	4	Janet	Gates
5	5	Lucy	Harrington

	SalesOrderID	OrderDate	CustomerID
1	71774	2008-06-01 00:00:00.000	29847
2	71776	2008-06-01 00:00:00.000	30072
3	71780	2008-06-01 00:00:00.000	30113
4	71782	2008-06-01 00:00:00.000	29485
5	71783	2008-06-01 00:00:00.000	29957

	ProductID	Name	ListPrice
1	680	HL Road Frame - Black, 58	1431,50
2	706	HL Road Frame - Red, 58	1431,50
3	707	Sport-100 Helmet, Red	34,99
4	708	Sport-100 Helmet, Black	34,99
5	709	Mountain Bike Socks, M	9,50

Consulta ejecutada correctamente. | localhost (17.0 RTM) | sa (94) | AdventureWorksLT | 00:00:00 | Fila: 1, Columna: 1 | 15 filas

Evidencia: verificación de conectividad y consulta de las tablas base Customer, SalesOrderHeader y Product.

2. Abstracción y Simplificación con Vistas (SalesLT.vCustomerOrders)

Creación de una vista relacional para encapsular la complejidad de uniones `INNER JOIN` entre clientes y órdenes de compra, estandarizando el formato del nombre mediante `CONCAT()` y exponiendo una interfaz lógica desacoplada de la estructura subyacente.

```sql
USE AdventureWorksLT;
GO

CREATE OR ALTER VIEW SalesLT.vCustomerOrders AS
SELECT
    c.CustomerID,
    CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
    h.SalesOrderID,
    h.OrderDate
FROM SalesLT.Customer c
INNER JOIN SalesLT.SalesOrderHeader h ON c.CustomerID = h.CustomerID;
GO

-- Consulta de validación
SELECT TOP (5) *
FROM SalesLT.vCustomerOrders
ORDER BY OrderDate DESC;
```

RESULTADO

Resultados | Mensajes

	CustomerID	CustomerName	SalesOrderID	OrderDate
1	29847	David Hodgson	71774	2008-06-01 00:00:00.000
2	30072	Andrea Thomsen	71776	2008-06-01 00:00:00.000
3	30113	Raja Venugopal	71780	2008-06-01 00:00:00.000
4	29485	Catherine Abel	71782	2008-06-01 00:00:00.000
5	29957	Kevin Liu	71783	2008-06-01 00:00:00.000

Consulta ejecutada correctamente. | localhost (17.0 RTM) | sa (94) | AdventureWorksLT | 00:00:00 | Fila: 1, Columna: 1 | 5 filas

Evidencia: resultado de la consulta de validación sobre la vista SalesLT.vCustomerOrders.

3. Procedimiento Almacenado Transaccional (dbo.AddOrderLineItem)

Implementación de un Stored Procedure para gestionar la inserción de líneas de pedido (`SalesOrderDetail`) y el recálculo automático del subtotal en la cabecera (`SalesOrderHeader`). La lógica opera bajo control transaccional explícito (`BEGIN TRANSACTION` / `COMMIT TRANSACTION`), validación de integridad referencial y emisión de errores personalizados con `THROW` .

```sql
USE AdventureWorksLT;
GO

CREATE OR ALTER PROCEDURE dbo.AddOrderLineItem
    @SalesOrderID INT,
    @ProductID    INT,
    @Quantity     INT
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRANSACTION;

    DECLARE @UnitPrice DECIMAL(18,2);
    SELECT @UnitPrice = CAST(ListPrice AS DECIMAL(18,2))
    FROM SalesLT.Product
    WHERE ProductID = @ProductID;

    -- Validación 1: Existencia del producto
    IF @UnitPrice IS NULL
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50010, 'Invalid ProductID specified.', 1;
    END

    -- Validación 2: Existencia del pedido
    IF NOT EXISTS (SELECT 1 FROM SalesLT.SalesOrderHeader WHERE SalesOrderID = @SalesOrderID)
    BEGIN
        ROLLBACK TRANSACTION;
        THROW 50011, 'Invalid SalesOrderID specified.', 1;
    END

    -- Inserción de la línea de detalle
    INSERT INTO SalesLT.SalesOrderDetail (SalesOrderID, OrderQty, ProductID, UnitPrice, UnitPriceDisc
    VALUES (@SalesOrderID, @Quantity, @ProductID, @UnitPrice, 0);

    -- Recálculo sincronizado del SubTotal en la cabecera
    UPDATE h
    SET SubTotal = d.SumLineTotal,
        ModifiedDate = SYSUTCDATETIME()
    FROM SalesLT.SalesOrderHeader h
    INNER JOIN (
        SELECT SalesOrderID, SUM(LineTotal) AS SumLineTotal
        FROM SalesLT.SalesOrderDetail
        WHERE SalesOrderID = @SalesOrderID
```

```sql
        GROUP BY SalesOrderID
    ) d ON d.SalesOrderID = h.SalesOrderID;

    COMMIT TRANSACTION;
END;
GO
```

EJECUCIÓN Y PRUEBA

```sql
DECLARE @SalesOrderID INT = (SELECT TOP 1 SalesOrderID
                             FROM SalesLT.SalesOrderHeader
                             ORDER BY SalesOrderID DESC);

EXEC dbo.AddOrderLineItem
    @SalesOrderID = @SalesOrderID,
    @ProductID = 680,
    @Quantity = 1;

SELECT TOP (5) *
FROM SalesLT.SalesOrderDetail
WHERE SalesOrderID = @SalesOrderID
ORDER BY SalesOrderDetailID DESC;

SELECT SalesOrderID, SubTotal, TaxAmt, Freight, TotalDue
FROM SalesLT.SalesOrderHeader
WHERE SalesOrderID = @SalesOrderID;
```

RESULTADO

Resultados | Mensajes

	SalesOrderID	SalesOrderDetailID	OrderQty	ProductID	UnitPrice	UnitPriceDiscount	LineTotal	rowguid	ModifiedDate
1	71946	113407	1	680	1431,50	0,00	1431.500000	E8385BC4-63B4-4F47-8CB6-3BBF01B739DE	2026-09-22 12:43:54.760
2	71946	113406	1	916	31,584	0,00	31.584000	B571CEE0-A90A-4E76-9796-379B55352A28	2008-06-01 00:00:00.000

	SalesOrderID	SubTotal	TaxAmt	Freight	TotalDue
1	71946	1463,084	3,1163	0,9738	1467,1741

Consulta ejecutada correctamente. | localhost (17.0 RTM) | sa (94) | AdventureWorksLT | 00:00:00 | Fila: 1, Columna: 1 | 3 filas

Evidencia: ejecución del procedimiento almacenado dbo.AddOrderLineItem y verificación del recálculo del subtotal.

4. Función Escalar Determinista (dbo.fnOrderTotal)

Desarrollo de una Scalar Function para computar el valor acumulado de una orden a partir de sus líneas de detalle, devolviendo un único valor de tipo `DECIMAL(18,2)`. Incluye `ISNULL()` para blindar el retorno ante pedidos sin líneas asociadas.

```sql
USE AdventureWorksLT;
GO

CREATE OR ALTER FUNCTION dbo.fnOrderTotal (@OrderID INT)
RETURNS DECIMAL(18,2)
AS
BEGIN
    DECLARE @Total DECIMAL(18,2);

    SELECT @Total = SUM(LineTotal)
    FROM SalesLT.SalesOrderDetail
    WHERE SalesOrderID = @OrderID;

    RETURN ISNULL(@Total, 0.00);
END;
GO

-- Invocación dentro de una cláusula SELECT agrupada
SELECT d.SalesOrderID, dbo.fnOrderTotal(d.SalesOrderID) AS OrderTotal
FROM SalesLT.SalesOrderDetail d
GROUP BY d.SalesOrderID
ORDER BY d.SalesOrderID DESC;
```

RESULTADO

Resultados | Mensajes

	SalesOrderID	OrderTotal
1	71946	1463.08
2	71938	74160.23
3	71936	79589.62
4	71935	5533.87
5	71923	96.11
6	71920	2527.13
7	71917	37.76
8	71915	1732.89
9	71902	59894.21
10	71899	1856.21
11	71898	53248.69
12	71897	10585.05
13	71895	221.26
14	71885	524.66
15	71867	858.90
16	71863	2777.14
17	71858	11528.84
18	71856	500.30

Consulta ejecutada correctamente. | localhost (17.0 RTM) | sa (94) | AdventureWorksLT | 00:00:00 | Fila: 1, Columna: 1 | 32 filas

Evidencia: prueba de la función escalar dbo.fnOrderTotal aplicada sobre las líneas de detalle.

5. Inline Table-Valued Function (ITVF) y Operador CROSS APPLY

Implementación de una función con valores de tabla en línea (`dbo.GetCustomerOrders`) que acepta parámetros y devuelve un result set tabular. Al ser de tipo Inline (una única instrucción `RETURN (SELECT ...)`), el optimizador expande la consulta directamente en el plan de ejecución, permitiendo la correlación fila por fila mediante `CROSS APPLY` .

```sql
USE AdventureWorksLT;
GO

CREATE OR ALTER FUNCTION dbo.GetCustomerOrders (@CustomerID INT)
RETURNS TABLE
AS
RETURN
(
    SELECT
        h.SalesOrderID,
        h.OrderDate
    FROM SalesLT.SalesOrderHeader h
    WHERE h.CustomerID = @CustomerID
);
GO

-- Consulta correlacionada fila por fila con CROSS APPLY
SELECT
    CONCAT(c.FirstName, ' ', c.LastName) AS CustomerName,
    o.SalesOrderID,
    o.OrderDate
FROM SalesLT.Customer c
CROSS APPLY dbo.GetCustomerOrders(c.CustomerID) o
WHERE c.CustomerID = 29929;
```

RESULTADO

	SalesOrderID	OrderDate
1	71902	2008-06-01 00:00:00.000

	CustomerName	SalesOrderID	OrderDate
1	Jeffrey Kurtz	71902	2008-06-01 00:00:00.000

Consulta ejecutada correctamente. | localhost (17.0 RTM) | sa (94) | AdventureWorksLT | 00:00:00 | Fila: 1, Columna: 1 | 2 filas

Evidencia: uso de la función de tabla en línea dbo.GetCustomerOrders mediante CROSS APPLY.

6. Disparador DML para Auditoría Automática (SalesLT.trg_LogOrderTotalChange)

Diseño de una tabla de auditoría (`dbo.OrderAudit`) y un Trigger `AFTER INSERT, UPDATE` sobre la tabla de detalle. El mecanismo intercepta modificaciones de cantidad, computa los totales previos y posteriores accediendo a las pseudotablas `deleted` e `inserted`, y registra la trazabilidad con marca de tiempo UTC (`SYSUTCDATETIME()`).

```
USE AdventureWorksLT;
GO

-- 1. Estructura de la tabla de auditoría
IF OBJECT_ID('dbo.OrderAudit') IS NULL
BEGIN
    CREATE TABLE dbo.OrderAudit (
        AuditID     INT IDENTITY(1,1) PRIMARY KEY,
        OrderID     INT NOT NULL,
        OldTotal    DECIMAL(18,2) NULL,
        NewTotal    DECIMAL(18,2) NULL,
        ChangedAt   DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
    );
END;
GO

-- 2. Definición del trigger DML
CREATE OR ALTER TRIGGER SalesLT.trg_LogOrderTotalChange
ON SalesLT.SalesOrderDetail
AFTER INSERT, UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    ;WITH AffectedOrders AS (
        SELECT SalesOrderID FROM inserted
        UNION
        SELECT SalesOrderID FROM deleted
    ),
    NewTotals AS (
        SELECT d.SalesOrderID, SUM(d.OrderQty * d.UnitPrice) AS Total
        FROM SalesLT.SalesOrderDetail d
        INNER JOIN AffectedOrders a ON d.SalesOrderID = a.SalesOrderID
        GROUP BY d.SalesOrderID
    ),
    InsertedTotals AS (
        SELECT SalesOrderID, SUM(OrderQty * UnitPrice) AS Total
        FROM inserted
        GROUP BY SalesOrderID
    ),
    DeletedTotals AS (
        SELECT SalesOrderID, SUM(OrderQty * UnitPrice) AS Total
        FROM deleted
        GROUP BY SalesOrderID
    )
```

```sql
    INSERT INTO dbo.OrderAudit (OrderID, OldTotal, NewTotal)
    SELECT
        n.SalesOrderID,
        n.Total - ISNULL(i.Total, 0) + ISNULL(d.Total, 0) AS OldTotal,
        n.Total AS NewTotal
    FROM NewTotals n
    LEFT JOIN InsertedTotals i ON n.SalesOrderID = i.SalesOrderID
    LEFT JOIN DeletedTotals d ON n.SalesOrderID = d.SalesOrderID;
END;
GO
```

PRUEBA DML Y VERIFICACIÓN DEL LOG

```sql
-- Provocar la actualización para disparar el trigger
UPDATE d
SET OrderQty = OrderQty + 1
FROM SalesLT.SalesOrderDetail d
WHERE d.SalesOrderID = (SELECT TOP 1 SalesOrderID FROM SalesLT.SalesOrderHeader ORDER BY SalesOrderID

-- Consultar la auditoría generada
SELECT TOP (5) *
FROM dbo.OrderAudit
ORDER BY AuditID DESC;
```

RESULTADO

	AuditID	OrderID	OldTotal	NewTotal	ChangedAt
1	1	71946	1463.08	2926.17	2026-09-22 10:51:06.5433728

Consulta ejecutada correctamente. | localhost (17.0 RTM) | sa (94) | AdventureWorksLT | 00:00:00 | Fila: 1, Columna: 1 | 1 filas

Evidencia: registro de auditoría generado por el trigger SalesLT.trg_LogOrderTotalChange tras la actualización DML.

Consideraciones Arquitectónicas (DP-800 Alignment)

- **Vistas:** se deben evitar vistas anidadas sobre otras vistas (*nested views*) para no entorpecer las estimaciones de cardinalidad del optimizador. Para modificaciones de datos con filtros obligatorios, se debe implementar `WITH CHECK OPTION`.
- **Procedimientos almacenados:** aprovechan el *Execution Plan Caching*. El uso de `SET NOCOUNT ON` es obligatorio para evitar tráfico innecesario en la red con mensajes de conteo de filas.
- **Funciones escalares vs. ITVFs:** las funciones escalares introducen penalizaciones de rendimiento por ejecución fila por fila (*Row-by-Row / RBAR*) al invocarse en filtros masivos. Siempre que sea posible, se deben priorizar las *Inline Table-Valued Functions*, tratadas por el optimizador como vistas parametrizadas.
- **Triggers:** diseñados exclusivamente para auditorías transparentes o sincronización entre tablas. Su lógica debe ser minimalista para no bloquear recursos dentro de la transacción original de escritura.